Lecture 8
Bash control structures (continued)

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 24, 2026

Last time: Exit status

- Any program, when it exits, gives a *status code* back to the OS to signal whether the command succeeded or failed
  - Also called *exit status*
- Code 0 means success, and any other code means failure
  - Commonly, code 1 is used for failure
- You an end a script by using the `exit` command, optionally specifying a status code, e.g. `exit 1`
  - The default code is 0.
- Each time we run a command, either interactively in the terminal, or via a script, the shell updates the variable $? with the exit status of the last command we ran.
  - Can check exit status of previous command with `echo $?`
    - Note: After you do this, the exit status will be replaced by that of the echo command!

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: If-statement

```
if COMMAND; then
    ...
elif COMMAND; then
    ...
else
    ...
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

Last time: While loop

```
while COMMAND; do
    ...
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Last time: `test` and `[`

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi


if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

Last time: For loop

```
for NAME in VALUE1 VALUE2 VALUE3
do
    COMMANDS
done
```

$NAME is set to VALUE1 on first iteration, VALUE2 on second iteration, etc.

Bérubé-Vallières © 2026
See last slide for complete credits

Reserved words (from POSIX documentation)

2.4 Reserved Words

Reserved words are words that have special meaning to the shell; see Shell Commands. The following words shall be recognized as reserved words:

!	**do**	**esac**	**in**
{	**done**	**fi**	**then**
}	**elif**	**for**	**until**
case	**else**	**if**	**while**

This recognition shall only occur when none of the characters is quoted and when the word is used as:

- The first word of a command
- The first word following one of the reserved words other than **case**, **for**, or **in**
- The third word in a **case** command (only **in** is valid in this case)
- The third word in a **for** command (only **in** and **do** are valid in this case)

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: `read`

- When we run a script from the terminal, it inherits all the file descriptors of the shell that runs the script.
  - Therefore, the script's stdin is connected to the terminal.
- Bash provides a built-in command called `read` that reads from stdin into a variable.
  - Reminder that you can find info about the built-in commands by typing `man builtins`
- Example:

```
echo "How old are you?"
read -r age
echo "You are $age years old"
```

exercise: use `man builtins` to find what –r does

Bérubé-Vallières © 2026
See last slide for complete credits

Example

- By also using a pipe, we can feed the output of a command into such a while loop, enabling us to loop over the lines of output produced by a program.

```
ls | while read -r file ; do
    if [ -d "$file" ] ; then
        echo "Is a directory: $file"
    else
        echo "Is not a directory: $file"
    fi
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Example

- This program asks the user if they want to proceed before either doing the thing, or exiting.

```
read -p "Are you sure? [y/N] " -r answer
if [ "$answer" != "y" ] ; then
    exit 0
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

`shift` example

- This script prints each command-line argument on its own line.

```
while [ $# -gt 0 ] ; do
    echo "$1"
    shift
done
```

- shift is a builtin command that "shifts" over all the numbered variables. That is, the value of $2 moves into $1, the value of $3 moves into $2, and so on. The value of $1 is lost.
- By using shift, we reduce the count of command-line argument by one, which is why $# eventually reaches zero, ending the loop.

Bérubé-Vallières © 2026
See last slide for complete credits

Short-circuiting using && and ||

- We previously saw that && executes the second command only if the first one exits successfully
  - e.g. mkdir my_dir && cd my_dir
    - # will only cd into my_dir if we created my_dir
- Can use this to chain conditions!
- Example:

```
if [ -f file1 ] && [ -f file2]; then
   echo "file1 and file2 exist“
else
   echo "missing file1 or file2!"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Short-circuiting using && and ||

- A similar operator is ||, which executes the second command only if the first one did *not* exit succesfully
- Can use this to chain conditions!
- Example:

```
if [ $1 = "cat" ] || [ $1 = "dog" ]; then
   echo "you selected cat or dog!"
else
   echo "you did not select cat or dog!"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Evaluating arithmetic expressions

- It is possible to evaluate arithmetic expressions in Bash by using $((expr))
- Examples:
  - `echo $(( 5 + 3 ))`
  - `result=$(( $1 + $2 ))`
    - What does `echo $result` output?
  - `echo $(( var++ ))`
    - Outputs `$var` and increases its value by 1 (post-increment)

Bérubé-Vallières © 2026
See last slide for complete credits

Evaluating arithmetic expressions (example with for loop)

```
i=1
weekdays="Mon Tue Wed Thu Fri"
for day in $weekdays
do
 echo "Weekday $((i++)) : $day"
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Evaluating arithmetic expressions (other example with for loop)

```
i=1
for file in /etc/[abcd]*.conf
do
 echo "File $((i++)) : $file"
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Case statement

```
case EXPRESSION in
   PATTERN1)
      ...
   ;;
   PATTERN2)
      ...
   ;;
   PATTERN3)
      …
   ;;
esac
```


Bérubé-Vallières © 2026
See last slide for complete credits

Case statement

```
case EXPRESSION in
   PATTERN1)
      ...
   ;;
   PATTERN2)
      ...
   ;;
   PATTERN3)
      …
   ;;
esac
```

if EXPRESSION matches a certain pattern, the corresponding block is executed

Bérubé-Vallières © 2026
See last slide for complete credits

Case statement (example)

```
animal=$1
case $animal in
   cat)
     echo "You entered 'cat'"
  ;;
  dog)
     echo "You entered 'dog'"
  ;;
  *)
     echo "You did not enter 'cat' or 'dog'"
  ;;
esac
```


Bérubé-Vallières © 2026
See last slide for complete credits

Case statement (example)

```
animal=$1
case $animal in
   *at)
     echo "You entered a word that ends with 'at'"
  ;;
  dog)
     echo "You entered 'dog'"
  ;;
  *)
     echo "You did not enter 'cat' or 'dog'"
  ;;
esac
```


Bérubé-Vallières © 2026
See last slide for complete credits

Case statement

```
case EXPRESSION in
   PATTERN1)
      ...
   ;;
   PATTERN2 | PATTERN3)
      ...
   ;;
   PATTERN3 | PATTERN4)
      …
   ;;
esac
```

Can use | when multiple patterns should execute the same block

Bérubé-Vallières © 2026
See last slide for complete credits

Case statement (example)

```
case $1 in
   add | addition)
     result=$(($2 + $3))
  ;;
  sub | subtraction)
     result=$(($2 - $3))
  ;;
  *)
     result=0
  ;;
esac
echo "The result is $result"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Case statement (example)

```
echo -n "Do you agree with this? [yes or no]: "
read yno
case $yno in
        [yY] | [yY][Ee][Ss] )
                echo "Agreed"
                ;;

        [nN] | [nN][Oo] )
                echo "Not agreed, you can't proceed the installation";
                exit 1
                ;;
        *) echo "Invalid input"
            ;;
esac
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions

- Like "real" programming languages, Bash has functions
  - However, implementation is limited
- Useful whenever there is repetitive code
- Syntax:

```
function_name() {
    commands…
}
```

- Functions are called by invoking their names, equivalent to calling a command

Bérubé-Vallières © 2026
See last slide for complete credits

Functions

- Like "real" programming languages, Bash has functions
  - However, implementation is limited
- Useful whenever there is repetitive code
- Syntax:

```
function_name() {
    commands…
}
```

Other syntax:

```
function function_name() {
    commands…
}
```

- Functions are called by invoking their names, equivalent to calling a command

Bérubé-Vallières © 2026
See last slide for complete credits

Functions (example)

```
#!/bin/bash
hello_world() {
    echo "Hello World!"
}

hello_world # calls hello_world()
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: Passing arguments

- Like a command, arguments can be passed to functions, and they can be accessed through `$1, $2, …`
- Example:

```
add() {
    echo $(( $1 + $2 ))
}

add 5 6 # will echo 11
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: Local variable

- A variable declared as local is only visible within the block of code in which it appears (local scope)
- Example:

```
fun() {
      local x=5
      y=6
}

fun
echo "Value of x outside function: $x"
# Will print "Value of x outside function:  " ($x is empty)
echo "Value of y outside function: $y"
# Will print "Value of y outside function: 6"
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: returning value

- Just like commands, functions return an exit status
  - 0 for success, non-zero for failure (can use `return` to specify)
  - Cannot trivially return a value as in “real” programming languages
- Can use command substitution and echo to return a value

```
add() {
     local sum=$(( $1 + $2 ))
     echo $sum
}

result=$(add 5 6) # result is now 11
```


Bérubé-Vallières © 2026
See last slide for complete credits

Functions: returning value

- Just like commands, functions return an exit status
  - 0 for success, non-zero for failure
  - Cannot trivially return a value as in “real” programming languages
- Can use command substitution and echo to return a value

```
add() {
      local sum=$(( $1 + $2 ))
      echo $sum
}

result=$(add 5 6) # result is now 11
```

- There are other methods, e.g using global variables, but can be error-prone

Bérubé-Vallières © 2026
See last slide for complete credits

More on text stream processing

Bérubé-Vallières © 2026
See last slide for complete credits

Text stream processing commands (1/3)

- `cut -d <delimiter> [file]`
  - `cut` command is used to extract specific sections of text from each line of a file or standard input. It works well with structured text (like columns or delimited data). You can select output by `-f` to extract fields separated by a delimiter
  - Example: `cut -d ',' -f 1,3 data.csv`
- `wc [-clw] [file]`
  - `wc` (word count) simply counts the number of words, lines, and characters in the file(s).
  - The three options, `clw`, stand for character, line, and word respectively, and tell `wc` which of the three to count.
- Example: `wc -l data.csv`

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- CSV = Comma Separated Values
- Text file with a specific format:
  - data1,data2,data3,data4
  - data5,data6,data5,data8
- File is composed of units of data that are separated from each other by commas or newlines
- CSV files are used as:
  - Log files
  - Configuration files
  - Sharing data between applications
  - Experiment output stream files
  - Simple databases

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- Each row is called a “record”
  - It stores data about a common artifact
- Each unit of data in a row is called a “field”
  - It stores a single fact related to the artifact with commas



- The above example stores information about people: first name, last name, age

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- Say that `users.csv` stores:

```
fname,lname,age
Mary,Brown,18
John,Smith,98
Mary,Johnson,24
```

- We can extract the lname column with `cut`

```
cut -d ',' -f2 users.csv
```

- Output:

```
lname
Brown
Smith
Johnson
```


Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- Say that `users.csv` stores:

```
fname,lname,age
Mary,Brown,18
John,Smith,98
Mary,Johnson,24
```

- We can extract the lname column with `cut`

```
cut -d ',' -f2 users.csv
```

- Output:

```
lname
Brown
Smith
Johnson
```

delimiter is a comma

second field

Bérubé-Vallières © 2026
See last slide for complete credits

`cut` example with CSV file

- Say that `users.csv` stores:

```
fname,lname,age
Mary,Brown,18
John,Smith,98
Mary,Johnson,24
```

- Ages of users with first name Mary

```
grep 'Mary' users.csv | cut -d ',' -f3
```

Output:

```
18
24
```


Bérubé-Vallières © 2026
See last slide for complete credits

Text stream processing commands (2/3)

- `sort [options] [file]`
  - `sort` command is used to sort data in file or standard input (in case of the absence of the file argument)
  - `-n` option treats sorting key as numeric (Important in ensuring 5 comes before 25, etc.)
  - `-r` sorts in descending order
  - `-o outputfile` writes the sort output to output file. Otherwise, sort will write to the standard output.
- `uniq [file]`
  - `uniq` (unique) command filters out repeated lines from a file or input stream. It removes consecutive duplicate lines and prints only one instance of each.
  - `-c` option prefix each line with the number of occurrences
  - **Important:** uniq only works properly on sorted input, because it only compares adjacent lines

Bérubé-Vallières © 2026
See last slide for complete credits

Text stream processing commands (3/3)

- Text stream processing commands are often chained together with pipes (|) to build powerful text-processing workflows.
- Example:

```
cat data.txt | cut -f2 | sort | uniq -c | sort -nr
```


Bérubé-Vallières © 2026
See last slide for complete credits

grep

- `grep` is used to search for the patterns in files.
- `grep [options] STRING FILE_LIST`
- `STRING` above is the pattern to match, and it is called a regular expression (or regex). We are not covering regex in this course, but here is a great resource if you would like to learn more about it: https://regexone.com/
- Some common options include:
  - `-i` : ignore case
  - `-n` : display the line number along with the line on which a match was found
  - `-c` : report only a count of the number of lines containing matches
  - `-v` : invert the search, displaying only lines that do not match
  - `-l` : list filenames, but not lines, in which matches were foun

Bérubé-Vallières © 2026
See last slide for complete credits

grep: example

- Consider the following content of file `students.txt`:

Alex
Marc
Michael
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem

```
$ grep 'Je' students.txt
Jeremy
Jessica
Jean-Sebastien
```


Bérubé-Vallières © 2026
See last slide for complete credits

grep: example

- Consider the following content of file `students.txt`:

Alex
Marc
Michael
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem

```
$ cat students.txt | grep 'Je'
Jeremy
Jessica
Jean-Sebastien
```


Bérubé-Vallières © 2026
See last slide for complete credits

grep: example

- Consider the following content of file `students.txt`:

Alex
Marc
Michael
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem

```
$ grep -n 'Je' students.txt
6:Jeremy
7:Jessica
10:Jean-Sebastien
```


Bérubé-Vallières © 2026
See last slide for complete credits

grep: example

- Consider the following content of file `students.txt`:

Alex
Marc
Michael
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem

```
$ grep -c 'Je' students.txt
3
```


Bérubé-Vallières © 2026
See last slide for complete credits

Quiet

- In case it's undesirable to see the output of a command run inside a script, there are a few ways to suppress a command's output:
  - Some commands come with flags to suppress their output, e.g. `grep -q` (the q stands for "quiet").
  - Redirect the output into `/dev/null` -- this is a special file that doesn't actually store any data and just ignores anything we try to write to it
    - Example: `ls > /dev/null 2> /dev/null`
      - Redirects stdout and stderr to /dev/null
    - Equivalently: `ls > /dev/null 2>&1`
      - Redirects stdout to /dev/null, and redirects stderr to stdout

Bérubé-Vallières © 2026
See last slide for complete credits

Reminder: Class Test 1

- October 6, during class time
- Four rooms:
  - Leacock 132
  - Leacock Room 232
  - McMed 521
  - McMed 1345
- I will announce where each student is assigned soon
- Exam is 75 minutes (1h15)
- Letter-sized (8.5” by 11”) handwritten crib sheet, single or double-sided
- Non-programmable calculator allowed
- First part of the final exam will be a retake of this midterm
- Practice exam posted, and TAs are writing additional practice problems

Bérubé-Vallières © 2026
See last slide for complete credits

Reminder

- No quiz next lecture
  - Next quiz is on Oct 8!

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides/notes from Prof. Vybihal and Prof. Errington

Bérubé-Vallières © 2026
See last slide for complete credits